

Database Architecture Report: grcPortal SQLite Implementation

Executive Summary

The grcPortal application implements a comprehensive SQLite database architecture using SQLAlchemy ORM for Governance, Risk, and Compliance (GRC) management. This report details how database tables are created, mapped to SQLite, and structured to support enterprise-grade security and compliance workflows.

Database Configuration

SQLite Database Setup

- Database Engine: SQLite 3.x
- Database File Location: `./instance/app.db` (Flask instance path)
- ORM Framework: SQLAlchemy 2.0+ with future compatibility

Connection Configuration:

```
py 9+ db.py x
py > get_session
from sqlalchemy.orm import sessionmaker

def get_engine():
    """
    Create and configure SQLAlchemy database engine.

    Creates a SQLite engine instance configured for the Flask application pattern.
    The database file is located at ./instance/app.db relative to the application.

    Returns:
        Engine: SQLAlchemy engine instance configured for SQLite database

    Note:
        Uses future=True for SQLAlchemy 2.0 compatibility
        Echo is disabled to prevent SQL query logging in production
    """
    # SQLite database in ./instance/app.db (Flask pattern)
    db_url = "sqlite:///instance/app.db"
    engine = create_engine(db_url, echo=False, future=True)
    return engine
```

Database Initialization Process

1. Engine Creation: SQLAlchemy engine instantiated with SQLite URL
2. Table Creation: Automatic table generation using `Base.metadata.create_all(engine)`
3. Session Management: Thread-safe database sessions via Flask application context
4. Migration Support: Alembic integration for schema evolution

Table Creation Methodology

SQLAlchemy Declarative Base

All database tables inherit from a common `Base` class:

```

py 9+  models.py 4 X
els.py > User

class Base(DeclarativeBase):
    """
    Base class for all SQLAlchemy database models.
    Provides common functionality and configuration for all database entities
    in the GRC portal. All models inherit from this base class to ensure
    consistent behavior and automatic timestamp management.
    Features:
    - Automatic table naming from class names
    - Declarative base for SQLAlchemy ORM
    - Foundation for relationship definitions
    - Consistent model structure across the application
    Note:
    This class should not be instantiated directly. Use specific model
    classes that inherit from Base for database operations.
    """
    pass

class User(Base):
    """...
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    email: Mapped[str] = mapped_column(String(255), unique=True, nullable=False)
    password_hash: Mapped[str] = mapped_column(String(255), nullable=False)

```

Table Definition Pattern

Tables are defined using SQLAlchemy's declarative syntax:

```

py 9+  models.py 4 X
dels.py > ScanResult

class ScanResult(Base):
    """...
    __tablename__ = "scan_results"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    upload_id: Mapped[int] = mapped_column(ForeignKey("uploads.id"), nullable=False)
    summary: Mapped[str] = mapped_column(Text, nullable=True)
    compliance_hits_json: Mapped[str] = mapped_column(Text, nullable=True) # JS
    risks_json: Mapped[str] = mapped_column(Text, nullable=True)
    scanned_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.now(tz=UTC))

    upload: Mapped["Upload"] = relationship("Upload", back_populates="scan_result")
    risks = relationship("Risk", back_populates="scan_result")

class Risk(Base):
    """Represents a comprehensive risk assessment in the system following NIST RMF
    __tablename__ = "risks"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    asset: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF:
    threat: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF:
    vulnerability: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF:
    control: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF:
    compliance_standard: Mapped[ComplianceFramework] = mapped_column(Enum(ComplianceFramework), nullable=False)
    status: Mapped[RiskStatus] = mapped_column(Enum(RiskStatus), default=RiskStatus.Pending)

```

Automatic SQLite Mapping

SQLAlchemy automatically maps Python data types to SQLite column types:

- Integer → INTEGER
- String(length) → VARCHAR(length)
- Text → TEXT
- Boolean → INTEGER (0/1 in SQLite)
- Float → REAL
- DateTime → TEXT (ISO format in SQLite)
- Enum → VARCHAR with CHECK constraints

Core Database Tables

1. Users Table (users)

Purpose: User account management and authentication

Creation Method: Declarative model with SQLAlchemy

```
class User(Base):
    """..."""
    __tablename__ = "users"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    email: Mapped[str] = mapped_column(String(255), unique=True, nullable=False)
    password_hash: Mapped[str] = mapped_column(String(255), nullable=False)
    is_verified: Mapped[bool] = mapped_column(Boolean, default=False)
    role: Mapped[str] = mapped_column(String(50), default="user") # user, admin, auditor
    governance_role: Mapped[GovernanceRole] = mapped_column(Enum(GovernanceRole), nullable=True)
    department: Mapped[str] = mapped_column(String(100), nullable=True)
    approval_limit: Mapped[float] = mapped_column(Float, default=0.0) # Maximum risk value user
    escalation_threshold: Mapped[int] = mapped_column(Integer, default=15) # Risk score threshold
    escalation_level: Mapped[str] = mapped_column(String(50), default="business_unit") # business
    audit_trail_enabled: Mapped[bool] = mapped_column(Boolean, default=True) # Enable audit logg

    created_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.
```

SQLite Mapping:

```
CREATE TABLE users (
    id INTEGER NOT NULL,
    email VARCHAR(255) NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    is_verified BOOLEAN NOT NULL,
    role VARCHAR(50) NOT NULL,
    governance_role VARCHAR(18),
    department VARCHAR(100),
    approval_limit FLOAT NOT NULL,
    escalation_threshold INTEGER NOT NULL,
    escalation_level VARCHAR(50) NOT NULL,
    audit_trail_enabled BOOLEAN NOT NULL,
    created_at DATETIME NOT NULL,
    updated_at DATETIME NOT NULL,
    PRIMARY KEY (id),
    UNIQUE (email)
);
```

2. Risks Table (`risks`)

Purpose: Comprehensive risk assessment storage

Features: Multi-criteria scoring, quantitative analysis

```
class Risk(Base):
    """Represents a comprehensive risk assessment in the system following NIST RMF and ISO 31000.
    __tablename__ = "risks"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    asset: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF: Information or system
    threat: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF: Potential for misuse
    vulnerability: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF: Weakness
    control: Mapped[str] = mapped_column(String(500), nullable=False) # NIST RMF: Safeguard/control
    compliance_standard: Mapped[ComplianceFramework] = mapped_column(Enum(ComplianceFramework), nullable=False)
    status: Mapped[RiskStatus] = mapped_column(Enum(RiskStatus), default=RiskStatus.OPEN)
    category: Mapped[RiskCategory] = mapped_column(Enum(RiskCategory), nullable=True)

    likelihood: Mapped[int] = mapped_column(Integer, default=1) # 1-5 scale
    impact: Mapped[int] = mapped_column(Integer, default=1) # 1-5 scale
    score: Mapped[int] = mapped_column(Integer, default=0) # likelihood x impact

    # Security classification for Bell-LaPadula enforcement
    severity: Mapped[RiskSeverity] = mapped_column(Enum(RiskSeverity), default=RiskSeverity.MEDIUM)

    ale: Mapped[float] = mapped_column(Float, default=0.0) # Annualized Loss Expectancy
    emv: Mapped[float] = mapped_column(Float, default=0.0) # Expected Monetary Value

    mitigation_plan: Mapped[str] = mapped_column(Text, nullable=True)
    residual_risk: Mapped[str] = mapped_column(Text, nullable=True)
```

SQLite Mapping:

```
CREATE TABLE risks (
    id INTEGER NOT NULL,
    asset VARCHAR(500) NOT NULL,
    threat VARCHAR(500) NOT NULL,
    vulnerability VARCHAR(500) NOT NULL,
    control VARCHAR(500) NOT NULL,
    compliance_standard VARCHAR(14),
    status VARCHAR(9) NOT NULL,
    category VARCHAR(24),
    likelihood INTEGER NOT NULL,
    impact INTEGER NOT NULL,
    score INTEGER NOT NULL,
    severity VARCHAR(8) NOT NULL,
    ale FLOAT NOT NULL,
    emv FLOAT NOT NULL,
    mitigation_plan TEXT,
    residual_risk TEXT,
    owner VARCHAR(255),
    treatment VARCHAR(8),
    rmf_phase VARCHAR(10) NOT NULL.
);
```

3. Incidents Table (incidents)

Purpose: Security incident tracking and IRP management

```
class Incident(Base):
    """Represents security incidents reported in the system."""
    __tablename__ = "incidents"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    title: Mapped[str] = mapped_column(String(255), nullable=False)
    description: Mapped[str] = mapped_column(Text, nullable=False)
    status: Mapped[IncidentStatus] = mapped_column(Enum(IncidentStatus), default=IncidentSta
    severity: Mapped[IncidentSeverity] = mapped_column(Enum(IncidentSeverity), default=Incid

    reported_by: Mapped[int] = mapped_column(ForeignKey("users.id"), nullable=False)
    reported_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(time
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(time

    # IRP Step Notes
    preparation_notes: Mapped[str] = mapped_column(Text, nullable=True)
    identification_notes: Mapped[str] = mapped_column(Text, nullable=True)
    containment_notes: Mapped[str] = mapped_column(Text, nullable=True)
    eradication_notes: Mapped[str] = mapped_column(Text, nullable=True)
    recovery_notes: Mapped[str] = mapped_column(Text, nullable=True)
```

SQLite Mapping:

```
CREATE TABLE incidents (
    id INTEGER NOT NULL,
    title VARCHAR(255) NOT NULL,
    description TEXT NOT NULL,
    status VARCHAR(10) NOT NULL,
    severity VARCHAR(8) NOT NULL,
    reported_by INTEGER NOT NULL,
    reported_at DATETIME NOT NULL,
    updated_at DATETIME NOT NULL,
    preparation_notes TEXT,
    identification_notes TEXT,
    containment_notes TEXT,
    eradication_notes TEXT,
    recovery_notes TEXT,
    PRIMARY KEY (id),
    FOREIGN KEY(reported_by) REFERENCES users (id)
);
```

4. Compliance Scores Table (compliance_scores)

Purpose: Framework compliance assessment storage

```
class Compliance(Base):
    """...
    __tablename__ = "compliance_scores"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    framework: Mapped[str] = mapped_column(String(255), nullable=False)
    control: Mapped[str] = mapped_column(String(255), nullable=False)
    control_family: Mapped[str] = mapped_column(String(100), nullable=True) # e.g., AC, IR, AU
    score: Mapped[float] = mapped_column(Float, default=0.0) # percentage compliance 0-
    status: Mapped[str] = mapped_column(String(50), default="not_assessed") # compliant, non-com
    automated_score: Mapped[float] = mapped_column(Float, default=0.0) # Automated calculated
    manual_override: Mapped[bool] = mapped_column(Boolean, default=False) # Whether score was ma
    assessment_method: Mapped[str] = mapped_column(String(100), default="manual") # manual, auto

    risk_id: Mapped[int] = mapped_column(Integer, ForeignKey("risks.id"), nullable=True)
    risk = relationship("Risk", back_populates="compliance")

    created_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.
    updated_at: Mapped[datetime] = mapped_column(DateTime, default=lambda: datetime.now(timezone.
```

SQLite Mapping:

```
CREATE TABLE compliance_scores (
    id INTEGER NOT NULL,
    framework VARCHAR(255) NOT NULL,
    control VARCHAR(255) NOT NULL,
    control_family VARCHAR(100),
    score FLOAT NOT NULL,
    status VARCHAR(50) NOT NULL,
    automated_score FLOAT NOT NULL,
    manual_override BOOLEAN NOT NULL,
    assessment_method VARCHAR(100) NOT NULL,
    risk_id INTEGER,
    created_at DATETIME NOT NULL,
    updated_at DATETIME NOT NULL,
    PRIMARY KEY (id),
    FOREIGN KEY(risk_id) REFERENCES risks (id)
);
```

Data Seeding and Population

Populating **user** database table

```
cur.execute("""
INSERT INTO users (
    email, password_hash, is_verified, role,
    approval_limit, escalation_threshold, escalation_level, audit_trail_enabled,
    created_at, updated_at
) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?)
""", (
    email,
    hashed_pw,
    True, # is_verified
    "user", # role
    10000.0, # approval_limit
    15, # escalation_threshold
    "business_unit", # escalation_level
    True, # audit_trail_enabled
    datetime.now(timezone.utc), # created_at
    datetime.now(timezone.utc) # updated_at
))
```

Table: users													
	id	email	password_hash	is_verified	role	governance_role	department	approval_limit	escalation_threshold	escalation_level	audit_trail_enabled	created_at	updated_at
1	1	kush786srj@gmail.com	script:...	1	admin	NULL	NULL	100000.0	15	executive	1	2025-10-06 ...	2025-10-06...
2	2	admin@example.com	script:...	1	auditor	NULL	NULL	10000.0	15	business_unit	1	2025-10-08 ...	2025-10-08...
3	3	saroj24@gmail.com	script:...	1	user	NULL	NULL	10000.0	15	business_unit	1	2025-10-08 ...	2025-10-08...

Populating **indicators_of_compromise** database table

```
conn.execute(text("""
INSERT INTO indicators_of_compromise (indicator_type, indicator_value, confidence, severity, status,
    threat_actor, campaign, malware_family, first_seen, last_seen, detection_source, description, tags,
    created_by, created_at, updated_at)
VALUES (:indicator_type, :indicator_value, :confidence, :severity, :status,
    :threat_actor, :campaign, :malware_family, :first_seen, :last_seen, :detection_source, :description,
    :tags, :created_by, :created_at, :updated_at)"""),
{
    "indicator_type": ioc_data["indicator_type"],
    "indicator_value": ioc_data["indicator_value"],
    "confidence": ioc_data["confidence"],
    "severity": ioc_data["severity"],
    "status": ioc_data["status"],
    "threat_actor": ioc_data.get("threat_actor"),
    "campaign": ioc_data.get("campaign"),
    "malware_family": ioc_data.get("malware_family"),
    "first_seen": datetime.now(timezone.utc) - timedelta(days=30),
    "last_seen": datetime.now(timezone.utc),
    "detection_source": "Internal Analysis",
    "description": ioc_data["description"],
    "tags": json.dumps(ioc_data["tags"]),
    "created_by": 1, # Default admin user
    "created_at": datetime.now(timezone.utc),
    "updated_at": datetime.now(timezone.utc)
})
```


SQL 1*

1

SELECT * from indicators_of_compromise;

	id	indicator_type	indicator_value	confidence	severity	status	threat_actor	campaign	malware_family	first_seen
1	1	ip	192.168.1.100	85	high	active	APT28	SolarWinds	NULL	2025-09-06 18:19:34.237687+00:00
2	2	domain	malicious.example.com	90	critical	active	Lazarus Group	Banking Malware	NULL	2025-09-06 18:19:34.238644+00:00
3	3	hash	a665a45920422f9d417e4867efdc4fb8a04a...	95	critical	active	Unknown	NULL	Ransomware	2025-09-06 18:19:34.239233+00:00
4	4	url	https://fake-bank-login.com	80	high	active	Unknown	Credential Theft	NULL	2025-09-06 18:19:34.239618+00:00
5	5	email	support@fake-bank.com	75	medium	active	Unknown	BEC	NULL	2025-09-06 18:19:34.240337+00:00
6	6	ip	192.168.1.100	85	high	active	APT28	SolarWinds	NULL	2024-01-15 00:00:00.000000

Execution finished without errors.

Result: 6 rows returned in 13ms

At line 1:

SELECT * from indicators_of_compromise;

DESCRIBE Table structure

Schema of “users” database table (through SQLite user interface)

SQL 1*						
1 PRAGMA table_info(users);						
	cid	name	type	notnull	dflt_value	pk
1	0	id	INTEGER	1	NULL	1
2	1	email	VARCHAR(255)	1	NULL	0
3	2	password_hash	VARCHAR(255)	1	NULL	0
4	3	is_verified	BOOLEAN	1	NULL	0
5	4	role	VARCHAR(50)	1	NULL	0
6	5	governance_role	VARCHAR(18)	0	NULL	0
7	6	department	VARCHAR(100)	0	NULL	0
8	7	approval_limit	FLOAT	1	NULL	0
9	8	escalation_threshold	INTEGER	1	NULL	0
10	9	escalation_level	VARCHAR(50)	1	NULL	0
11	10	audit_trail_enabled	BOOLEAN	1	NULL	0
12	11	created_at	DATETIME	1	NULL	0
13	12	updated_at	DATETIME	1	NULL	0

Schema of “incidents” database table (through SQLite user interface)

SQL 1*						
1 PRAGMA table_info(incidents);						
	cid	name	type	notnull	dfbt_value	pk
1	0	id	INTEGER	1	NULL	1
2	1	title	VARCHAR (255)	1	NULL	0
3	2	description	TEXT	1	NULL	0
4	3	status	VARCHAR (10)	1	NULL	0
5	4	severity	VARCHAR (8)	1	NULL	0
6	5	reported_by	INTEGER	1	NULL	0
7	6	reported_at	DATETIME	1	NULL	0
8	7	updated_at	DATETIME	1	NULL	0
9	8	preparation_notes	TEXT	0	NULL	0
10	9	identification_notes	TEXT	0	NULL	0
11	10	containment_notes	TEXT	0	NULL	0
12	11	eradication_notes	TEXT	0	NULL	0
13	12	recovery_notes	TEXT	0	NULL	0

SQLite-Specific Optimizations

Data Type Handling

- Booleans: Stored as INTEGER (0/1) in SQLite
- Enums: VARCHAR with application-level validation
- JSON Fields: TEXT columns with JSON serialization
- Timestamps: ISO 8601 formatted TEXT strings

Performance Considerations

- Connection Pooling: SQLAlchemy handles SQLite connections efficiently
- Transaction Management: Explicit commits for data integrity
- Indexing: Automatic indexing on primary and foreign keys
- Concurrency: SQLite WAL mode for better concurrent access

SQLite Security Features

- File-based Security: Database file permissions
- Encryption Ready: Can be extended with SQLCipher
- Backup Strategy: Application-level export/import capabilities

Conclusion

The grcPortal database architecture successfully maps complex GRC requirements to SQLite through SQLAlchemy ORM. The implementation provides:

- Scalability: 40+ tables supporting comprehensive GRC workflows
- Performance: Efficient SQLite operations with proper indexing
- Maintainability: Declarative models with migration support
- Security: Role-based access and audit capabilities

The database serves as the foundation for enterprise GRC management, enabling risk assessment, compliance monitoring, incident response, and threat intelligence integration within a single, cohesive SQLite-backed system.